# Text Priors Transfer Little: Label Sharing and Test-Pool Calibration for Cold-Start Predictive AI Evaluation

**Anonymous team**
Technical report for the NeurIPS 2026 Predictive AI Evaluation Competition (draft)

## Abstract

The Predictive AI Evaluation Competition (PAIEC) scores predicted probabilities that an AI system answers a task correctly, on private benchmarks, first without labels and then with a few revealed labels per system and benchmark. We report four findings from building our entry. First, a prior learned from task and system text with a frozen multilingual encoder (E5) on 166 public benchmarks transfers almost nothing to held-out benchmarks, and its score, a weighted area under the Brier learning curve (ALC), stays within 0.13% of a Beta-binomial baseline that uses only the revealed labels. Second, sharing labels across systems on the same benchmark and across identical or similar items lowers ALC by 1.82% in a simulator of the official draws. Third, the official practice-phase (formative) test pool is more extreme than the training data. The feedback of a constant-prediction baseline reveals each group's distance from 50% accuracy, and reweighting simulated groups to match it moves the simulated score of this baseline from 0.2136 to 0.1826, against 0.1778 on the platform. Under this calibration a wider prior and priors from release date and model name help, and our final model lowers ALC by 4.1% (standard error 0.14%) on fresh simulated draws of recent systems. A Beta prior tuned to this pool recovers less than a tenth of that gain, and without label sharing across systems the final model loses 0.0031 of its 0.0072 ALC gain. Fourth, official runs of one baseline spread by 0.0493 over 6 draws and the leaderboard keeps each team's best run, so we compare submissions by repeated runs.

## 1 Introduction

In PAIEC a submission maps a description of an AI system (the *subject*) and a task (the *item*) to a probability of a correct answer [1]. The test benchmarks are private, so every prediction starts cold at the benchmark level. For each subject and benchmark pair (a *group*) the platform splits items into an acquisition pool and an evaluation pool and reveals acquisition labels at budgets of 0, 1, 3, 7, 15 and 31 labels. Each evaluation run scores a random *draw* of about nine groups (Section 2).

The obvious approach learns a prior from the text of items and subject descriptions on public benchmarks and adapts it to the revealed labels. Our earlier entries V2 and V3 followed it with frozen E5 embeddings and four or five training benchmarks [2]. On its official draw, V2 had a Brier score of 0.2536 before any label was revealed, worse than the 0.25 of a constant prediction of 0.5. We asked whether more data fixes this and trained on 166 binary benchmarks of measurement-db, about 36 times more recorded responses than V3. It did not. In leave-benchmark-out simulation the best prior weight $\gamma$ was the smallest value we tried, and ALC matched the label-only Beta-binomial baseline within 0.13% (Section 7.1).

What helped was sharing label evidence (Section 4). The *level model* fits a benchmark offset from the labels of every subject in the draw. A same-item shift uses labels of other subjects on the same item,

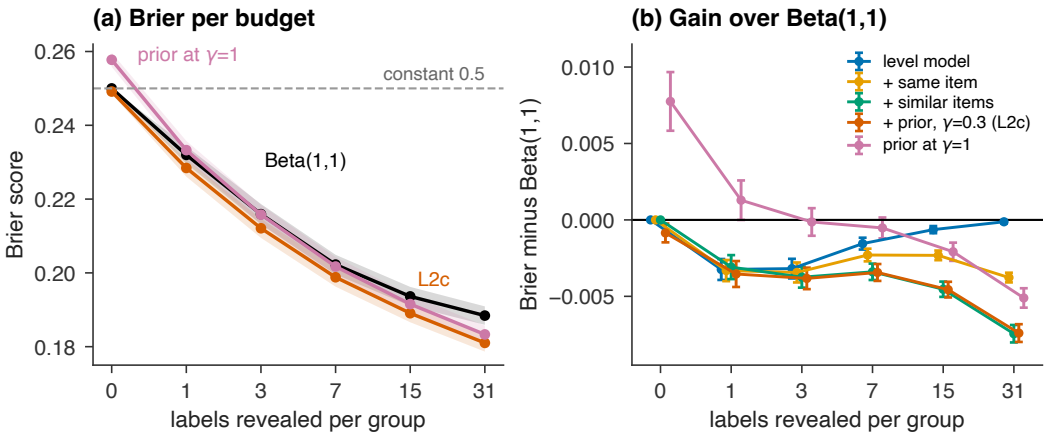



Figure 1: **On the training distribution, sharing labels across subjects and items carries the gain, and the text prior helps only at budget 0.** Simulator without test weights, all subjects, 600 fresh draws (4,823 groups), narrow level prior. (a) Mean Brier score per budget for Beta-binomial with a Beta(1,1) prior, the full narrow model (L2c) and L2c with the prior at full weight ($\gamma = 1$). (b) Paired difference from Beta(1,1) as each part is added. Bands and bars are 95% intervals over draws.

and a similar-item shift uses labels of items with close E5 embeddings. On fresh simulated draws this lowered ALC by 0.00388 (1.82%), and 95% of the gain remained without the text prior (Figure 1, Section 7.2).

Our simulator first missed the official level badly. It gave the official Beta-binomial baseline an ALC of 0.2136, while three official runs averaged 0.1778. The reason is the test pool. Because the baseline predicts one constant per group, its official Brier and calibration scores reveal every group's distance from 50% accuracy, and test groups are far more extreme than training groups (Section 5). Reweighting simulated groups to this distribution reproduces the official baseline curve (Figure 2a). It also changes the best model. The narrow prior chosen before is no better than Beta-binomial under these weights, while a wider prior and offsets from release date and model name help. Our final model, L5, lowers ALC by 4.1% on fresh draws of recent subjects and is first or close to first in both subject populations we test (Section 7.3). The best Beta prior for this pool gains only 0.0006, so the gain does not come from prior shape alone.

Official runs cannot confirm differences of this size one at a time. Our 6 runs of the Beta-binomial baseline spread by 0.0493, and the public leaderboard keeps each team's best run (Section 7.4). Our contributions are the following.

1. A measured negative result for frozen-encoder text priors, with two encoders and three training variants on 166 benchmarks (Section 7.1).
2. A label-sharing adaptation model that carries most of the attainable gain (Sections 4.2 and 7.2).
3. A calibration of the simulator to the formative test pool from aggregate feedback statistics, which predicts the official baseline level and changes model selection (Sections 5 and 7.3).
4. A repeated-run protocol for formative evaluation with a power calculation and an estimate of the best-of-$N$ bias (Section 7.4).

## 2 Task, score and the structure of official draws

A subject has eight string fields such as model name, harness and reasoning effort. An item has its content, metadata features, interactors and an anonymous benchmark alias. A submission's `predict([subject, item], labeled)` returns a probability, and `labeled` holds every label revealed so far in the draw, for all groups. For group $g$ with Brier score $B_g(k)$ after $k$ revealed labels,

$$\mathrm{ALC}_g = 0.1\, B_g(0) + 0.2\, \big[B_g(1) + B_g(3) + B_g(7) + B_g(15)\big] + 0.1\, B_g(31), \tag{1}$$

Table 1: **V5 trains on about 36 times more recorded responses than V3.** Groups are subject-benchmark pairs. Accuracies are benchmark-weighted means over groups with at least 80 items.

| | V3 training data | V5 corpus |
|---|---|---|
| Benchmarks | 5 | 166 |
| Recorded binary responses | 1,142,294 | 40,775,314 |
| Subject-item pairs and items | | 8,810,555 and 329,666 |
| Subject configurations (with responses) | | 14,453 (11,420) |
| Groups with at least 80 items | 82 (all sizes) | 8,765 on 150 benchmarks |
| Mean group accuracy, all subjects | | 0.510 (55% above 0.5) |
| Mean group accuracy, released from July 2025 | | 0.614 (68% above 0.5) |

the normalized trapezoidal area under the Brier curve over $\log_2(1+k)$, and the score is the unweighted mean of $\mathrm{ALC}_g$ over the groups in the draw. $B_g(k)$ averages squared errors over all recorded responses of the group's evaluation items.

Our 10 formative feedback tables show the draw structure (Appendix D). Of these, 8 contain 9 groups on 7 benchmarks with 2 benchmarks contributing 2 subjects each, and 2 contain 8 groups on 7 benchmarks. The evaluation pool is half of a group's items and holds 44 to 76 items in all but 2 group evaluations, so most groups have about 88 to 152 items. All 88 group evaluations came from the same 7 benchmark aliases, and the curation guideline states that test subjects are systems from 2025 and 2026 [1].

Three assumptions link our simulations to this setting.

A1. Held-out public benchmarks with recent subjects, reweighted to the test pool's accuracy distribution (Section 5), stand in for the private benchmarks. If they differ in other ways, simulated gains need not transfer, in either direction.

A2. Two subjects drawn on one benchmark share most of their items, because the platform takes whole groups. The feedback reports only counts, so we cannot verify this. If it fails, the same-item shift has no effect.

A3. A revealed label is one recorded response of the item. Our simulators draw it as a Bernoulli variable with the item's empirical success rate, which has the same distribution.

## 3 Data

All training data come from the measurement-db dataset at revision `54074453` [3], which is part of the competition's permitted inventory. Of its 281 benchmarks, 173 have binary responses. We did not download 4 of them (mind2web, naturalreasoning, swe_smith, tensortrust), which have one to three subjects each and include two with more than 500,000 items, and 3 more had no binary rows. The remaining 166 benchmarks form the corpus (Table 1). Each benchmark keeps at most 4,000 items, the ones answered by the most subjects, and we store the number of recorded trials $n$ and successes $k$ of every subject-item pair. A subject is the exact tuple of its eight fields, so one base model under another harness is a different subject. Subject descriptions are sparse. Of the 14,453 configurations, 45% have a model name, 31% a release date and 0.2% a reasoning effort.

## 4 Method

### 4.1 Text prior

Items and subjects are embedded with the frozen `intfloat/multilingual-e5-small` encoder [6] at revision `614241f`, with the recipe of V3 (Appendix B). An item text is its metadata line followed by its task, and a subject text lists its fields. A prior network gives a base logit $z_{si}$ for subject $s$ and item $i$. Two towers, each a two-layer perceptron with 256 Gaussian error linear units, map the subject input (E5 vector and release date) and the item input (E5 vector and text length) to $h_s$ and $h_i$. The logit is $z_{si} = a(h_s) + b(h_i) + \langle Uh_s, Vh_i\rangle/\sqrt{32}$ with linear heads $a$, $b$ and rank-32 projections $U$, $V$. Training minimizes the binomial log loss of the counts $(n, k)$ of every pair, with weight $1/(nT_g)$

for a pair in group $g$ with $T_g$ trials. Each benchmark gets a free offset during training, and the mean offset is folded into the intercept afterwards.

### 4.2 Label-sharing adaptation

All labels in `labeled` that carry the target's benchmark alias enter the update, whichever subject they belong to. For one benchmark, the logit of subject $s$ on item $i$ is

$$\eta_{si} = \gamma z_{si} + o_s + c + d_s + u_i, \qquad c \sim \mathcal{N}(0, \tau_c^2), \quad d_s \sim \mathcal{N}(0, \tau_d^2), \tag{2}$$

where $\gamma$ scales the text prior and $o_s$ is a fixed subject offset (Section 4.3). The benchmark offset $c$ is shared by all subjects in the draw, $d_s$ is the offset of subject $s$, and $u_i$ is an item shift. We fit $c$ and all $d_s$ jointly by Newton's method and take a Laplace approximation, which gives the posterior mean $m_s$ and variance $v_s$ of $c + d_s$. A label of another subject moves $c$ and therefore the target's prediction. We call this part with $u_i = 0$ the *level model*.

The item shift uses residuals of the labeled pairs under the level model. For labeled pair $j$ with subject $s_j$ and item $i_j$, let $\hat{p}_j = \sigma(\gamma z_j + o_{s_j} + m_{s_j})$, $r_j = y_j - \hat{p}_j$ and $w_j = \hat{p}_j(1 - \hat{p}_j)$. For a kernel $K$ with prior variance $\sigma_K^2$,

$$u_i = \sum_K \frac{\sigma_K^2 \sum_j K(i, i_j)\, r_j}{1 + \sigma_K^2 \sum_j K(i, i_j)\, w_j}, \tag{3}$$

which is one Newton step from zero for an offset shared by the target item and the labeled items, weighted by $K$. The same-item kernel $K(i, i') = \mathbf{1}[i = i']$ with variance $\sigma_e^2$ transfers labels of the same item from other subjects. The similar-item kernel $K(i, i') = \exp\big((\cos(e_i, e_{i'}) - 1)/h\big)$ on E5 vectors, with bandwidth $h$ and variance $\sigma_k^2$, transfers labels of near-duplicate items. With $\eta^*_{si} = \gamma z_{si} + o_s + m_s + u_i$, the prediction $p_{si} = \sigma\big(\eta^*_{si} / \sqrt{1 + \pi v_s / 8}\big)$ integrates the posterior variance with the probit approximation [10].

### 4.3 Release-date and model-name offsets

The fixed offset $o_s$ adds two subject priors fitted on training groups with at least 80 items. The date prior is $-0.472 + 0.285\, t$, with $t$ the years from 2023-01-01 to the release date clipped to $[-3, 4]$, and $-0.093$ for an unknown date, fitted by benchmark-weighted least squares to group logit accuracy. The name ability of a normalized model name is the mean, over its training groups, of the group logit minus the benchmark's mean logit, shrunk with a pseudo-count of 2 and clipped to $\pm 2.5$. It enters with weight 0.8, and the submission holds 683 names. In simulation the name ability comes from the four other benchmark folds only. There, it has a positive within-benchmark Spearman correlation with group accuracy on 71% of 62 held-out benchmarks with at least five named groups (median 0.18).

### 4.4 Final submission

The final archive, L5, uses $\tau_c = 2.5$, $\tau_d = 1.0$, $\sigma_e^2 = 1$, $h = 0.03$, $\sigma_k^2 = 3$, $\gamma = 0.3$, the date prior and the name ability with weight 0.8. It holds the E5 weights, the prior network, the name table and the code (282.6 megabytes) and declares no `models.txt`. It runs on CPU with one prediction worker, which the platform restarts at every budget, so the encoder runs again for each budget. A draw of 8 groups then takes 8 to 20 minutes on a two-vCPU Colab CPU runtime (Appendix A). Caches of embeddings, logits and level fits are keyed by content hashes, so no prediction depends on earlier calls. No predictor reads any group identifier.

## 5 Calibrating the simulator to the formative test pool

For a group whose prediction is one constant $q$, the Brier score is $p(1-p) + (q-p)^2$ and the expected calibration error (ECE) is $|q - p|$, where $p$ is the accuracy of the group's evaluation responses. The official Beta-binomial baseline predicts such a constant at every budget, namely 0.5 at budget 0. Its feedback therefore gives $|p - 0.5| = \mathrm{ECE}_0$ for every group, and $p(1-p) = B_{31} - \mathrm{ECE}_{31}^2$ agrees with it to $10^{-6}$.

Three baseline runs give 26 group evaluations. Their mean $p(1-p)$ is 0.136, against 0.184 for training groups, and the shares of $|p-0.5|$ in the bins $[0, 0.1)$, $[0.1, 0.2)$, $[0.2, 0.3)$, $[0.3, 0.4)$ and $[0.4, 0.5]$ are 0.115, 0.077, 0.231, 0.308, 0.269. So 58% of test groups have accuracy below 0.2 or above 0.8. The side of 0.5 is not identified, because flipping every label maps $p$ to $1-p$ and $q$ to $1-q$ and leaves both scores unchanged, so the feedback says nothing about the direction of a shift. The upward shift of our date prior rests on training data and the curation guideline alone. Training groups of subjects released from July 2025 have mean accuracy 0.614, 68% of them lie above 0.5 (Table 1), and the guideline states that test base models are 2025 and 2026 releases.

*Test weights* reweight simulated groups so that their $|p-0.5|$ bins match these shares. The tuning grids used an earlier estimate (0.115, 0.077, 0.192, 0.308, 0.308) that placed one group in a different bin, and all confirmation runs use the shares above. With test weights the simulated official baseline, Beta(2,2), scores 0.1826 on all subjects and 0.1841 on recent subjects, against 0.2136 and 0.2105 without them. The three official runs average 0.1778 (standard error 0.0049), and the calibrated curve follows the official one at every budget (Figure 2a). Later runs, made after the model choice and not used for the weights, scored 0.2190, 0.1777, 0.1835. The matching curve is a consistency check, not an independent test. Once the shares of $|p-0.5|$ match, the curve of a constant-per-group baseline follows mostly from group sizes and the split, so it does not validate the ranking of models. Single official runs of V2 (0.1695) and V3 (0.1711) lie within one run's standard deviation (0.0174) of the replay prediction that both score close to Beta-binomial (Section 7.3). Only these aggregate statistics enter model selection.

## 6 Evaluation protocols

**Baseline.** The Beta-binomial baseline predicts $(k+a)/(m+2a)$ for every evaluation item of a group with $m$ revealed labels and $k$ successes. Simulations use $a=1$ and the official archive uses $a=2$, which is worse by 0.00023 ALC in simulation (standard error 0.00019).

**Leave-benchmark-out simulation.** We split the 166 benchmarks into 5 random folds, train the prior on four and simulate on groups of the fifth with at least 80 items, subsampled to 80 to 160 items. Draws collect groups up to 1,000 items, 60 per fold, from 8,765 held-out groups. This protocol uses the level model only.

**Official-structure simulator.** A draw picks 2 benchmarks whose two sampled subjects share 88 to 152 items and 5 benchmarks with one subject, and drops groups beyond 1,000 items, so most draws hold 8 groups. Each group is split in half independently. Base logits come from the prior trained without the pair's benchmark fold. Options restrict subjects to those released from 2025 (56 benchmarks) and apply test weights. Seeds 0 to 599 serve development, and 600 seeds from 100,000, 200,000 and 300,000 serve one confirmation run each. Standard errors treat the draw as the unit.

**Runtime replays.** We ran the real V2 and V3 submission code on the simulator's draws (recent subjects, seeds from 0). Each group is paired with the simulator's L5 and Beta-binomial rows on the same draw. These draws exclude the nine benchmarks that V2 or V3 trained or tuned on (Appendix A).

**Formative runs.** Each Codabench submission is scored on a new random draw. We upload an archive once, resubmit it several times and report the mean and standard error across runs.

## 7 Results

### 7.1 The text prior transfers almost nothing to held-out benchmarks

In all three training variants the best prior weight was $\gamma = 0.3$, the smallest value in the grid, and ALC fell steadily as $\gamma$ shrank (Table 2). Variant B, with free benchmark offsets during training, reached 0.21217 against 0.21244 for Beta-binomial, a difference of $-0.13\%$. On a subset of at most 600 items per benchmark, Qwen3-Embedding-0.6B [7] gave 0.21651 against 0.21684 for E5 and 0.21801 for Beta-binomial. One set of draws per run gives no standard error, so we read the gap between encoders as no evidence either way. The prior moves mainly budget 0. At $\gamma = 0.3$ it lowered the simulated

Table 2: **In every variant the best prior weight is the smallest one tried.** Leave-benchmark-out ALC with the level model, best $(\tau_c, \tau_d)$ per row, lower is better. Each block has its own draws and baseline. Variant C adds dropout 0.3, weight decay 0.01 and 2 epochs to B.

| Data | Prior | ALC | vs Beta-binomial |
|---|---|---|---|
| Full corpus | Beta-binomial, Beta(1,1) | 0.21244 | |
| | B, $\gamma = 0.3$ | 0.21217 | $-0.13\%$ |
| | B, $\gamma$ from 0.45 to 0.9 | 0.21258 to 0.21503 | |
| | A (no benchmark offsets), $\gamma = 0.3$ | 0.21260 | $+0.08\%$ |
| | C (more regularization), $\gamma = 0.3$ | 0.21229 | $-0.07\%$ |
| 600-item subset | Beta-binomial, Beta(1,1) | 0.21801 | |
| | E5 prior, $\gamma = 0.3$ | 0.21684 | $-0.54\%$ |
| | Qwen3-Embedding-0.6B prior, $\gamma = 0.3$ | 0.21651 | $-0.68\%$ |

budget-0 Brier score from 0.25 to 0.2492, and at full weight it raised it to 0.2579 (Figure 1a), in line with V2's official 0.2536. A zero-shot LLM judge did no better. Difficulty ratings by Qwen3-8B [8] improved simulated ALC by only 0.00035 and hurt budget 0, so the judge is not used (Appendix C).

### 7.2 On the training distribution, shared label evidence carries the gain

Without test weights, the narrow model ($\tau_c = 1.2$, $\tau_d = 0.6$, $\sigma_e^2 = 2$) lowered ALC from 0.21260 for Beta-binomial to 0.20872. This paired difference of $-0.00388$ (standard error 0.00023) comes from 600 fresh draws of all subjects. Without its text prior it reached $-0.00369$, which is 95% of the gain. Each level of sharing helps at a different budget (Figure 1b). The benchmark offset helps most at 1 to 3 labels, when the target subject has few labels but the other subject on the benchmark adds more. The same-item shift grows with the budget, and the similar-item shift doubles the gain at 31 labels.

### 7.3 On the test pool, prior width and date and name offsets matter

Test weights change the ranking of models (Table 3). Without them the narrow model L2c is best in both subject populations, ahead of L5 by 0.0022 on all subjects. With them L5 beats L2c by 0.0024 on all subjects and 0.0024 on recent ones (standard errors 0.0002). Since only the weighted simulator reproduces the official baseline level, we read the unweighted ranking as a description of the training distribution, not of the test pool. The narrow level model with the same-item shift (L1c) is no better than Beta-binomial under test weights ($+0.0005$ on all subjects), although it gains 0.0029 without them. Extreme groups need large offsets, and a narrow prior holds their predictions too close to the starting value at low budgets.

On recent subjects with test weights, the wide model with item shifts and the E5 prior (L3) reaches $-0.0063$. The date and name offsets lower ALC by a further 0.0010 (standard error 0.0001), to $-0.0072$ for L5, or $-4.1\%$. The text prior accounts for 0.0007 (standard error 0.0001) of the gain, more than on the training distribution because budget 0 weighs more in extreme groups (Figure 2b).

Two checks show that the gain on the test pool is not only a prior that suits extreme groups. First, the best Beta prior for this pool among those we tried, Beta(0.5,0.5), improves on Beta(1,1) by only 0.0006 under test weights on recent subjects, and L5 beats it by 0.0066 (standard error 0.0003). The official Beta(2,2) prior is worse than Beta(1,1) by 0.0055. Second, L5 restricted to the labels of the target group loses 0.0031 (standard error 0.0002) of its 0.0072 gain, because it no longer shares the benchmark offset or same items across subjects. In the wide level model alone, sharing the benchmark offset between the two subjects of a benchmark is worth 0.0022 (standard error 0.0001). This part needs only two subjects on one benchmark, which the feedback confirms, and not assumption A2. The same-item step adds 0.0006.

We kept L5 over the tuned L6. Under test weights L5 is first or within 0.0002 of first in both populations. L6 is better by 0.0002 on recent subjects but worse by 0.0006 on all subjects (standard errors 0.0001), so it depends more on the recency restriction.

The runtime replays agree. With test weights, L5 beat the real V2 code by 0.0053 (standard error 0.0011, 120 draws, 974 groups) and V3 by 0.0053 (0.0010, 120 draws). Without weights the differences were $-0.0008$ (0.0009) and $-0.0001$ (0.0008). V2 was close to Beta-binomial under test

Table 3: **Under test weights the wide prior and the date and name offsets win, while without them the narrow model is best.** Official-structure simulator, the same 600 confirmation draws (seeds from 300,000) in each population. First row is the Beta(1,1) ALC, other rows are differences from it, lower is better. Rows with own labels only use the labels of the target group and share nothing across subjects. Standard errors are 0.0001 to 0.0003 per cell and near 0.0001 for paired differences between rows.

| Subjects | all | all | recent | recent |
|---|---|---|---|---|
| Test weights | no | yes | no | yes |
| Beta-binomial ALC | 0.2136 | 0.1769 | 0.2099 | 0.1786 |
| Beta-binomial, Beta(0.5,0.5) | +0.0043 | −0.0006 | +0.0037 | −0.0006 |
| Beta-binomial, Beta(2,2), official prior | +0.0000 | +0.0057 | +0.0006 | +0.0055 |
| L1c, narrow level + same item | −0.0029 | +0.0005 | −0.0033 | −0.0004 |
| L2c, narrow, all sharing + E5 prior | −0.0043 | −0.0027 | −0.0062 | −0.0049 |
| Wide level model | −0.0002 | −0.0027 | −0.0008 | −0.0032 |
| + same item | −0.0009 | −0.0032 | −0.0016 | −0.0038 |
| + similar items | −0.0021 | −0.0045 | −0.0032 | −0.0052 |
| + E5 prior, $\gamma = 0.3$ (L3) | −0.0023 | −0.0049 | −0.0039 | −0.0063 |
| + date prior (L4) | −0.0023 | −0.0052 | −0.0043 | −0.0068 |
| + name ability (L5, final) | −0.0021 | −0.0051 | −0.0044 | −0.0072 |
| L5 without the E5 prior | −0.0021 | −0.0049 | −0.0039 | −0.0065 |
| Wide level model, own labels only | +0.0017 | −0.0009 | +0.0013 | −0.0010 |
| L5, own labels only | +0.0010 | −0.0024 | −0.0008 | −0.0041 |
| L6, best of a grid on recent dev draws | −0.0023 | −0.0045 | −0.0051 | −0.0075 |

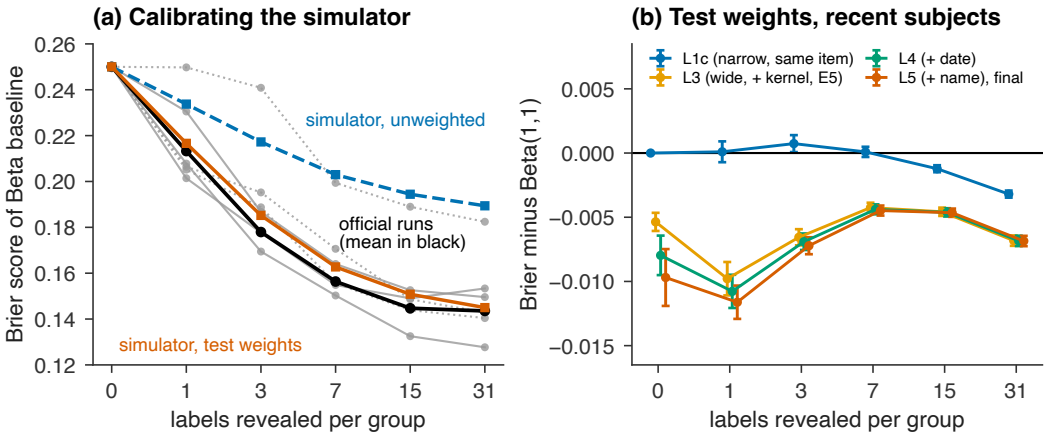



Figure 2: **Reweighting simulated groups to the test pool reproduces the official baseline curve, and under these weights L5 gains most at low budgets.** (a) Brier score per budget of the Beta(2,2) baseline in the three official runs that set the weights (grey, mean in black), in later runs (dotted) and in the simulator with and without test weights (all subjects, 600 confirmation draws). (b) Paired difference from Beta(1,1) under test weights, recent subjects, 600 confirmation draws. Bars are 95% intervals over draws.

weights (−0.0013, standard error 0.0013) and better without them (−0.0029). So V2 and V3 match L5 on the training distribution but fall behind it on the extreme test pool, where their narrow priors lose the gain over Beta-binomial. A replay of the L5 runtime itself on 3 draws (23 groups) matched the simulator within 0.0044 ALC per group on average.

### 7.4 Single formative runs cannot rank these methods

All 6 runs of the Beta(2,2) baseline archive scored 0.1867, 0.1698, 0.1769, 0.2190, 0.1777, 0.1835 (mean 0.1856, standard deviation 0.0174, Table 4), and public repositories of two other teams report standard deviations of 0.016 over 20 runs [4] and 0.024 over 4 runs [5]. Every run gets its own draw, so comparisons between archives are unpaired, and detecting a difference of 0.00388 with a

Table 4: **Formative scores are single draws or small samples.** Official Brier ALC per run, lower is better, each on its own draw of 8 or 9 groups. Not started means queued on 5 November 03:04 UTC.

| Archive | Runs | ALC per run | Mean (s.e.) |
|---|---|---|---|
| Beta-binomial, Beta(2,2) | 3 | 0.1867, 0.1698, 0.1769 | 0.1778 (0.0049) |
| Beta(2,2), runs after model selection | 3 | 0.2190, 0.1777, 0.1835 | 0.1934 (0.0129) |
| V2 | 1 (2 not started) | 0.1695 | |
| V3 | 1 | 0.1711 | |
| L5, final | 0 (4 not started) | none | |

two-sided 5% test and 80% power needs 268 runs per archive at a standard deviation of 0.016 and 316 at our 0.0174, against 18 paired simulated draws. The leaderboard shows each team's best run, which for 20 runs at a standard deviation of 0.016 lies about 0.030 below the mean, and one team reports a 20-run mean of 0.1765 and a leaderboard entry of 0.139 [4]. The final evaluation scores each team once on a common set, so the mean is the relevant quantity.

## 8 Related work

Item response theory models a correct answer through subject ability and item difficulty [11] and has been applied to NLP leaderboards [12]. tinyBenchmarks estimates a benchmark score from a few items with item parameters fitted on that benchmark [13], which a cold start lacks. Observational scaling laws predict scores from capabilities measured on known benchmarks [14], not item by item on unseen ones. Text-based difficulty estimation works within a domain [15], and amortized calibration predicts IRT difficulty from question content [18], the role of our text prior, which we find transfers little to new benchmarks. Assessors predict a system's success on an instance [19], and a generic assessor does so for a new model tested on about 100 reference instances [20], while PAIEC gives at most 31 labels, all from the new benchmark. Our level model is a random-intercept logistic model with partial pooling [16, 17]. None of these settings combines unseen benchmarks with a few labels per subject, which is the regime PAIEC scores.

## 9 Limitations

The main limit is the test-weight calibration. It rests on 26 group evaluations from three runs on the same 7 benchmark aliases, and the final private benchmarks may differ from them. It uses only $|p - 0.5|$, since the side of 0.5 is not identified. Model selection used these aggregate statistics, so the formative pool is not untouched for our choice of L5. Resampling the bins of the 26 evaluations (20,000 bootstrap replicates) keeps L5 ahead of Beta-binomial in every replicate (90% interval $-0.0089$ to $-0.0056$ on recent confirmation draws), but L5 beats the narrow L2c in only 94% of them ($-0.0049$ to $+0.0001$). The bootstrap ignores groups that recur across runs and so understates this uncertainty, as later Beta runs show. Their 27 evaluations are less extreme (mean $p(1-p)$ 0.151), and under their shares alone the gaps of L5 to Beta(1,1) and to L2c are $-0.0058$ and $-0.0006$. Under the shares of all 53 evaluations they are $-0.0065$ and $-0.0014$. If the final pool resembles training data more than the formative pool, L2c would score about 0.0022 better than L5 (Table 3).

The date prior was fitted on all training benchmarks, including those held out in simulation, which slightly favors L4 and L5 there, while the name ability is fold-specific. The same-item shift relies on assumption A2, and without shared items the gain is smaller by up to the same-item step (0.0006 under test weights). Confirmation draws correct for tuning but not for earlier design choices such as the simulator, and Appendix A lists smaller mismatches. Finally, the stalled platform queue left L5 without an official run by 5 November 03:04 UTC.

## 10 Conclusion

A frozen-encoder prior trained on 166 benchmarks changed ALC by 0.13% in leave-benchmark-out simulation, while sharing labels across subjects and items carried most of the gain, also against a Beta prior tuned to the test pool. The feedback of a constant-prediction baseline calibrated our simulator to the test pool and changed which model we submit.

## References


[1] Stanford AI Measurement Science. The Predictive AI Evaluation Competition at NeurIPS 2026. Rules, data policy, curation guideline and protocol, 2026. `https://aims.stanford.edu/competition`.

[2] Anonymous team. Frozen semantic priors with hierarchical evidence adaptation for predictive AI evaluation. Technical report draft for PAIEC, 20 October 2026. **[TODO public link]**.

[3] AIMS Foundations. measurement-db. Hugging Face dataset, revision 54074453, 2026. `https://huggingface.co/datasets/aims-foundations/measurement-db`.

[4] veseluhha. paiec-2026. Public GitHub repository, accessed 4 November 2026. `https://github.com/veseluhha/paiec-2026`.

[5] Shashwat Srivastav. s2wish-paiec2026. Public GitHub repository, accessed 4 November 2026. `https://github.com/Shashwat-srivastav/s2wish-paiec2026`.

[6] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual E5 text embeddings. A technical report. arXiv:2402.05672, 2024.

[7] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 Embedding. Advancing text embedding and reranking through foundation models. arXiv:2506.05176, 2025.

[8] An Yang and others (Qwen Team). Qwen3 technical report. arXiv:2505.09388, 2025.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[10] David J. C. MacKay. The evidence framework applied to classification networks. *Neural Computation*, 4(5):720-736, 1992.

[11] Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum, 1980.

[12] Pedro Rodriguez, Joe Barrow, Alexander Miserlis Hoyle, John P. Lalor, Robin Jia, and Jordan Boyd-Graber. Evaluation examples are not equally informative. How should that change NLP leaderboards? In *ACL*, 2021.

[13] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks. Evaluating LLMs with fewer examples. In *ICML*, 2024.

[14] Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. In *NeurIPS*, 2024.

[15] Luca Benedetto, Paolo Cremonesi, Andrew Caines, Paula Buttery, Andrea Cappelli, Andrea Giussani, and Roberto Turrin. A survey on recent approaches to question difficulty estimation from text. *ACM Computing Surveys*, 55(9), 2023.

[16] Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311-319, 1975.

[17] Andrew Gelman and Jennifer Hill. *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Cambridge University Press, 2007.

[18] Sang Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. In *ICML*, 2025.

[19] José Hernández-Orallo, Wout Schellaert, and Fernando Martínez-Plumed. Training on the test set. Mapping the system-problem space in AI. In *AAAI*, 2022.

[20] Lorenzo Pacchiardi, Lucy G. Cheke, and José Hernández-Orallo. 100 instances is all you need. Predicting the success of a new LLM on unseen data by testing on a few instances. arXiv:2409.03563, 2024.

[21] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS Datasets and Benchmarks*, 2023.

# A Reproduction

Paths are relative to the project root. All CPU steps, the simulator and the runtime tests ran on the Mac with Python 3.12.12, torch 2.14.1, transformers 5.19.0, numpy 2.5.3, pandas 3.0.6, pyarrow 25.0.1 and huggingface_hub 1.33.0. GPU steps ran on Google Colab L4 runtimes with their preinstalled torch and transformers, driven by the colab command-line tool 0.7.4. Two mismatches remain between training and runtime. The prior's training weights give groups with repeated trials less weight than the score does, and training embeddings were stored in float16 while the runtime uses float32. The corpus omits 4 large or single-subject benchmarks and keeps the most-answered items of large benchmarks (Section 3). Because in-memory caches last one budget, L5 encodes 405 to 588 texts per budget of a draw. On a two-vCPU Colab CPU runtime with one torch thread this costs 0.29 seconds per text and 4.5 seconds per model load, so a run reaches the 8-hour limit only at about 260 groups, against 8 or 9 in a formative run. **[TODO Add the public repository link.]**

**1. Download and build the corpus (local CPU).** The download fetches benchmark tables at the pinned revision and skips the four benchmarks listed in Section 3.

```
python - <<'EOF'
import json
from huggingface_hub import snapshot_download
R, REV = "aims-foundations/measurement-db", "54074453da7b9911be0f48e4c560be547bc84d21"
skip = {"mind2web", "naturalreasoning", "swe_smith", "tensortrust"}
keep = [b for b in json.load(open("colab_jobs/binary_benchmarks.json")) if b not in skip]
snapshot_download(R, repo_type="dataset", revision=REV, local_dir="data/mdb_full",
    allow_patterns=["*/formatted_tables/benchmarks.parquet", "*/formatted_tables/subjects.parquet"]
    + [f"{b}/formatted_tables/{t}.parquet" for b in keep for t in ("responses", "items")])
EOF
RAW=data/mdb_full OUT=data/corpus_full RUNTIME_DIR=v5 EMBED=0 SKIP_FETCH=1 python colab_jobs/build_corpus.
     py
```

**2. Embeddings, priors and held-out logits (one L4 GPU).** We run jobs on Colab through `colab_jobs/ctl.py` (budget check, upload, watch, download with SHA-256 check, stop) and `colab_jobs/supervisor.py` (status file, stall kill, release of idle machines). On the GPU machine, `/content/run` holds `v5/{spec,encoder,textproj,adapt}.py` and the job scripts, and `/content/work` holds the corpus.

```
python /content/run/embed_texts.py
python /content/run/train_eval.py cv --tag A
python /content/run/train_eval.py cv --tag B --bench-offset 1
python /content/run/train_eval.py cv --tag C --bench-offset 1 --dropout 0.3 --wd 0.01 --epochs 2
cd /content/run4 && RUNTIME_DIR=/content/run python export_z.py --tag B   # needs train_eval3.py in /
     content/run3
```

The last command writes `pairs_z.parquet` with held-out logits from five benchmark folds and `prior_B.npz`, the prior trained on all benchmarks (SHA-256 `486b104f...`). The encoder comparison runs `feat_bakeoff.py` and `train_eval2.py` with `-item-keys-file` on the 600-item subset. The subject offsets come from `python colab_jobs/fit_subject_priors.py`, which writes `date_prior.json`, `v5/name_ability.json` and `name_ability_folds.json` and reproduces our files exactly.

**3. Test-pool statistics and simulations (local CPU).** The first command computes the statistics of Section 5. The others reproduce Table 3, Figure 1, the runtime replays and the encoder cost. The baseline script reuses the ladder's draws and stops if they differ.

```
cd reports/paiec_v5_report
python -B scripts/test_pool_stats.py
python -B scripts/sim_ladder.py --tag recent_conf --min-date 2025-01-01 --seed0 300000  # also all_conf,
     and dev tags with --seed0 0
python -B scripts/sim_baselines.py --tag recent_conf --min-date 2025-01-01 --seed0 300000  # also all_conf
python -B scripts/sim_confirm.py --draws 600 --seed0 100000                               # Figure 1
python -B scripts/extra_stats.py                                                          # judge, name
     ability, replays
cd ../.. && EX=bertaqa,cellverse,cruxeval,engdesign,erbench,kernelbench,liveaopsbench,researchcodebench,
     tau2_bench
python -B colab_jobs/replay_runtime.py UNZIPPED_V2_DIR colab_jobs/results/replay_v2.parquet --seeds 0:120 \

  --min-date 2025-01-01 --exclude $EX
python -B reports/paiec_v5_report/scripts/record_encoder_texts.py UNZIPPED_L5_DIR texts.json
```

```
python reports/paiec_v5_report/scripts/time_encoder.py texts.json encode_timing.json  # CPU machine, next
     to v5/encoder.py
```

The last two commands give the encoder cost of Section 4.4. The first writes item texts from measurement-db, so its output is not in the repository. The ladder and confirmation scripts import frozen copies of the simulator (`scripts/sim_official_v2_snapshot.py`, SHA-256 `8da09880...`, and `scripts/sim_official_snapshot.py`). The paired simulator rows came from `sim_official.py -emb data/corpus_full/item_emb.npy -draws 120 -min-date 2025-01-01 -exclude $EX -dump colab_jobs/results/sim_dump_recent_ex.parquet` with the variants Beta, L1c, L3, L4 and L5. The dump stores raw per-group ALC and $p$, so test weights are applied afterwards.

**4. Final archive.** This command rebuilds L5 and checks it with the validator of the organizers' starting kit, which is not part of our repository.

```
P=colab_jobs/results/exportB/work/results
python v5/build_zip.py $P/prior_B.npz $P/prior_B.json 2.5 1.0 0.3 dist/v5/L5_wide_date_name.zip \
  '{"use_prior": true, "item_exact": 1.0, "item_kernel": [0.03, 3.0], "name_scale": 0.8, "date_prior": '"$
     (cat colab_jobs/results/date_prior.json)"'}'
python starting_kit/tools/check_submission_zip.py dist/v5/L5_wide_date_name.zip
```

Our L5 archive has SHA-256

`2d8b7034279191413724be277a60fd9ec057b4bf697d42b873790f839cb83760`.

The script `build_zip.py` takes the five E5 files from the V3 archive or, if it is absent, from the hub at the pinned revision, and checks their SHA-256. A rebuild gives identical file contents.

**5. Formative runs and this report.** The first three commands resubmit an uploaded archive, collect the scores of finished runs and record the runs still waiting in the queue. The others rebuild the numbers, the figures and this PDF.

```
python scripts/resubmit.py L5_wide_date_name.zip 3 --label L5
python scripts/track.py
python scripts/queue_snapshot.py
cd reports/paiec_v5_report && python -B scripts/make_numbers.py && python -B scripts/make_figures.py
pdflatex main.tex && pdflatex main.tex
```

## B Hyperparameters and prompts

**Text projection.** Each text gets the E5 query prefix, and hidden states are mean-pooled under the attention mask and normalized to unit length. Texts longer than 8,192 characters keep a head, a middle and a tail span. After tokenization, inputs longer than 510 tokens keep four spans of 128, 127, 127 and 128 tokens spread evenly over the text. A JSON chat transcript that ends in a user turn keeps up to 512 characters of system messages and the final user message. The item text is the first 300 characters of the item features followed by this projection. The subject text lists name, provider, release date, harness with version, reasoning effort and the first 300 characters of the extras.

**Prior.** Subject input is the E5 vector, the years from 2023 to the release date clipped to $\pm 5$ and a missing-date flag. Item input is the E5 vector and $\log(1 + \text{characters})/10$. Hidden width 256, rank 32, dropout 0.1, AdamW [9] with learning rate $2 \times 10^{-3}$ and weight decay $10^{-4}$, one-cycle schedule with 10% warm-up, batch 8,192, 4 epochs, seed 99 for the final prior and the fold index for fold priors.

**Leave-benchmark-out grid.** The grid crosses $\tau_c \in \{0.8, 1.2, 1.8, 2.5, 3.5\}$ with $\tau_d \in \{0.5, 0.8, 1.2\}$ and $\gamma \in \{0.3, 0.45, 0.6, 0.75, 0.9\}$, 75 configurations in total. The first run used $\gamma$ from 0.7 upward and was 1.32% worse than Beta-binomial.

**Simulator grids.** Newton iterations stop after 25 steps or when the largest step is below $10^{-7}$. All three grids of the official-structure simulator used 400 development draws and include the Beta(1,1) baseline. Here $\gamma = 0$ means no text prior. Grid 1 (no test weights, all subjects, 487 configurations) crossed $\tau_c \in \{0.9, 1.2, 1.6\}$, $\tau_d \in \{0.6, 0.8, 1.1\}$, $\sigma_e^2 \in \{2, 4, 8\}$, $h \in \{0.02, 0.03, 0.05\}$, $\sigma_k^2 \in$

$\{1, 1.5, 3\}$ and $\gamma \in \{0, 0.3\}$. Its best difference was $-0.00463$ (standard error 0.00028), and it chose the narrow configuration. Grid 2 (earlier test-weight shares, all subjects, 289 configurations) crossed $\tau_c \in \{2, 2.5, 3.5, 5\}$, $\tau_d \in \{0.8, 1, 1.4, 2\}$, $\sigma_e^2 \in \{1, 2, 4\}$, $\sigma_k^2 \in \{1.5, 3, 6\}$ at $h = 0.03$, and $\gamma \in \{0, 0.3\}$. It chose the wide level prior of L3. Grid 3 (earlier test-weight shares, subjects from 2025, 326 configurations with L5) crossed $\tau_c \in \{2, 2.5, 3.2\}$, $\tau_d \in \{0.8, 1, 1.3\}$, $\sigma_e^2 \in \{0.5, 1, 2\}$, $\gamma \in \{0.2, 0.3, 0.45\}$, name weight $\{0.8, 1.2\}$ and date weight $\{1, 1.5\}$ at $\sigma_k^2 = 3$ and $h = 0.03$. Its best configuration is L6. Top lines are in `colab_jobs/results/sim_testw_grid2.log` and `sim_grid3_recent.log`. Of the 600 draws of the unweighted confirmation run, 119 hold 9 groups, 385 hold 8 and 96 hold 7.

**Judge prompt.** The system message reads "You estimate how hard a task is for current frontier AI systems. Read the task and rate its difficulty on a scale from 1 to 9, where 1 means almost every AI system solves it correctly and 9 means almost none does. Answer with a single digit only." The user message is "Task:" followed by the item text, truncated to 1,024 tokens of head and tail, and "Difficulty (1-9):". Thinking mode is off.

## C The LLM difficulty judge

Following the use of LLMs as graders [21], Qwen3-8B [8] rated item difficulty from 1 to 9 as the expected digit of its next-token distribution, with the prompt of Appendix B. It rated 13,880 of 18,952 sampled items before we stopped the run to save compute, so long items are missing. The within-benchmark Spearman correlation between rating and item accuracy had median $-0.062$ and was negative, the useful direction, on 63% of 131 benchmarks. Between benchmarks it was $-0.049$. In the simulator on rated items with test weights, the best weight of 0.2 on the standardized rating improved ALC by only 0.00035 and hurt budget 0.

## D Formative runs

Table 5 lists every finished formative run of our team. V1 and the V3 variant with an uncertainty band are earlier entries not described here.

Table 5: **All finished formative runs.** Means over groups of one draw each. $B_0$ and $B_{31}$ are mean Brier scores at budgets 0 and 31.

| Submission | Archive | Groups | Benchmarks | ALC | $B_0$ | $B_{31}$ |
|---|---|---|---|---|---|---|
| 1003760 | Beta(2,2) baseline | 9 | 7 | 0.1867 | 0.2500 | 0.1496 |
| 1003791 | V1 hierarchical prior | 8 | 7 | 0.1911 | 0.2511 | 0.1612 |
| 1004759 | V2 | 9 | 7 | 0.1695 | 0.2536 | 0.1330 |
| 1004796 | V3 | 9 | 7 | 0.1711 | 0.2401 | 0.1478 |
| 1004970 | V3 with uncertainty band | 9 | 7 | 0.1876 | 0.2341 | 0.1644 |
| 1032881 | Beta(2,2) baseline | 9 | 7 | 0.1698 | 0.2500 | 0.1277 |
| 1032882 | Beta(2,2) baseline | 8 | 7 | 0.1769 | 0.2500 | 0.1533 |
| 1034630 | Beta(2,2) baseline | 9 | 7 | 0.2190 | 0.2500 | 0.1824 |
| 1034631 | Beta(2,2) baseline | 9 | 7 | 0.1777 | 0.2500 | 0.1404 |
| 1034661 | Beta(2,2) baseline | 9 | 7 | 0.1835 | 0.2500 | 0.1429 |

## E Training sources, pretrained models and compute

**Training sources.** All training and development data come from measurement-db at revision `54074453da7b9911be0f48e4c560be547bc84d21`, restricted to the 166 binary benchmarks of Section 3. No other data source was used for training, tuning or simulation. The test weights use aggregate statistics of our own formative feedback (Section 5). Earlier entries V2 and V3 used four and five benchmarks of the same inventory [2].

**Pretrained models.** No model was fine-tuned, and no paid API was used. Table 6 lists every pretrained model we ran.

Table 6: **Pretrained models used in this work.** Only E5 runs inside the submission.

| Model | Revision | Use |
|---|---|---|
| `intfloat/multilingual-e5-small` | `614241f622f53c4eeff9890bdc4f31cfecc418b3` | frozen encoder in training and in the submission |
| `Qwen/Qwen3-Embedding-0.6B` | `97b0c614be4d77ee51c0cef4e5f07c00f9eb65b3` | encoder comparison only |
| `Qwen/Qwen3-4B` | `1cfa9a7208912126459214e8b04321603b3df60c` | feature extraction, stopped before it finished |
| `Qwen/Qwen3-8B` | `b968826d9c46dd6066d109eabc6255188de91218` | difficulty judge |

**Compute.** GPU work ran on Google Colab L4 instances, and one Colab CPU runtime timed the encoder. Together they used 25.9 compute units of our own. Corpus building, simulation, replays, packaging and this report ran on an Apple M4 laptop.